

Privacy-First Worklet Model & Pipeline Architecture Report

AI/ML models used across the redaction pipeline, why each was chosen, and the full worker & workflow inventory

9

ML MODELS / MODEL
FAMILIES

12

WORKER SERVICES

11

END-TO-END FLOWS

Prepared for: Samsung PRISM Mentors

Project: Privacy-First Worklet (PRISM Privacy Pipeline)

Date: August 31, 2026

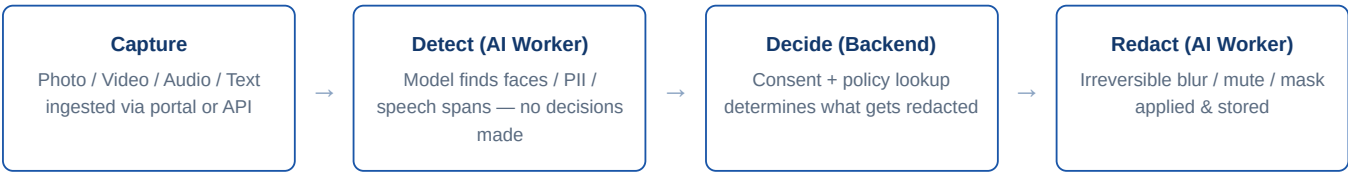
1. Executive Summary

PRISM is an on-device / self-hosted privacy pipeline that detects and redacts personally identifiable information (PII) — faces, voices, on-screen text, and document text — from photos, video, audio recordings, and text before they enter permanent storage. The system runs **zero data through third-party cloud AI APIs in its production redaction path**: every detection and redaction model executes locally, inside the project's own containers, which is the core design constraint behind almost every model choice below.

The pipeline is organised as **modality-specific microservices** (one per data type: face/image, image-PII/OCR, audio, text, video), each exposing a strict **detect** → **decide** → **redact** contract: the AI worker only detects and executes redaction; the Node.js backend is the sole place where consent is evaluated and a redaction decision is made. This separation is repeated consistently across every worker and is called out explicitly in the codebase as a deliberate security boundary, not an accident of implementation.

2. Architecture at a Glance

Every content type flows through the same three-stage shape:



Five standalone AI/ML microservices implement the "Detect" and "Redact" stages, one per modality. A Node.js backend owns every "Decide" stage plus seven background queue/timer workers that handle recognition dispatch, retries, retention, and Data-Subject-Rights (DSAR) execution. Redis/BullMQ and Qdrant tie the whole system together as the orchestration layer (Section 6).

3. AI/ML Models Used — By Pipeline Stage

Models are grouped by the modality/worker that owns them. Full rationale for each choice — taken directly from code comments, READMEs, and configuration — follows each table.

3.1 Image Pipeline — `face-worker` (port 8001) & `image-pii-worker` (port 8002)

Model	Task	Why this model was chosen
InsightFace buffalo_l (SCRFD detector + ArcFace, 512-d embeddings)	Face detection & recognition embeddings for clustering and bystander redaction	Strong open-source accuracy/speed balance; runs entirely on CPU because face-work happens as a post-session batch pass, not in a live request path, so GPU latency isn't needed. Package version is left unpinned deliberately for wheel-availability on newer Python. Embeddings are never persisted by this worker — computed, matched in memory by the backend, then discarded (data-minimisation by design).
RapidOCR (ONNX runtime)	Text extraction (OCR) from photos	Weights are bundled inside the Python wheel itself — no network download at build or run time, which keeps the container fully offline-capable and its build reproducible.
Microsoft Presidio + spaCy en_core_web_sm + custom India-specific regex recognizers	PII/NER detection on OCR'd text (Aadhaar, PAN, GSTIN, Voter ID, Passport, vehicle plates, UPI, IFSC, bank a/c, etc.)	Presidio's stock recognizer set has no coverage for Indian government ID formats, so the team layered ~15 custom validated regex recognizers on top. The service is fail-closed: any OCR/analysis error returns HTTP 503 instead of "no PII found," since a missed blur is treated as strictly worse than a false alarm.

3.2 Video Pipeline — `video-worker` (port 8005)

Model	Task	Why this model was chosen
InsightFace buffalo_l (same pack as face-worker) + custom IoU / constant-velocity tracker (non-ML)	Strided face detection, cross-frame tracking, sparse per-track embedding on video	Documented in code as a forced compatibility choice, not an independent one : the session's face gallery holds ArcFace vectors produced by face-worker, and embedding video faces with a different model would make cosine-similarity comparisons "confidently wrong." Detection is strided and embedding is sparse (5 frames/track) purely for throughput — a 2-minute clip is ~3,600 frames and doesn't need per-frame inference to track a face reliably.
<i>(delegates to image-pii-worker over HTTP)</i>	On-screen text PII detection in sampled video frames	Reuses one single, canonical definition of "what counts as PII" for both still photos and video instead of maintaining a second OCR/NER stack.

3.3 Audio Pipeline — `audio-worker` (port 8003)

Model	Task	Why this model was chosen
pyannote/speaker-diarization-3.1	Speaker diarization — splits audio into anonymous speaker turns	Widely-used, state-of-the-art open diarization pipeline; the gated Hugging Face license is documented and handled via an explicit token, and the model is loaded lazily so it never blocks the health-check endpoint.

	(who spoke when)	
faster-whisper ("small" size, configurable)	Speech-to-text transcription with word-level timestamps	Whisper size is an env-configurable speed/accuracy dial. Voice-activity-detection filtering is deliberately turned off so quiet or short bystander speech is never silently dropped before it can be scanned for PII.
SpeechBrain ECAPA-TDNN (<code>spkrec-ecapa-voxceleb</code>)	Speaker/voice embedding (192-d vector) for enrollment & identity matching	Compact (~80MB), open, well-established voice-embedding model — small enough to keep container size and cold-start time reasonable while remaining accurate for consent-matching.
Presidio (built-in recognizers)	PII detection on the transcript text	Spoken transcript text doesn't require the custom Indian-ID recognizers built for scanned documents, so the stock Presidio entity set (phone, email, name, location, date, etc.) is sufficient here.

3.4 Text / Document Pipeline — `text-worker` (port 8004)

Model	Task	Why this model was chosen
Microsoft Presidio + ~15 custom regex/rule recognizers (India ID formats, generic PII, secrets)	PII entity detection with character-level offsets, then consent-aware span redaction	The production endpoint is deliberately rule-based, not NER-model-based : Presidio's default phone/ID recognizers don't cover Indian formats (Aadhaar, PAN, GSTIN, UPI, IFSC, vehicle registration), so validated, checksum-aware custom recognizers replace them. This keeps the production redaction path fully deterministic and auditable — no model non-determinism in a compliance-critical decision.
spaCy <code>en_core_web_sm</code>	Tokenization/NER — <i>demo tool only</i>	Used only by an offline Streamlit demo, not by the deployed FastAPI redaction service.
(optional, demo-only) OpenAI GPT (<code>gpt-5.4-nano</code>)	Fallback disambiguation for hard person-name cases	Explicitly excluded from the production path — gated behind an API key and confined to the exploratory Streamlit tool, preserving the "no PII ever leaves the pipeline to a third party" guarantee for the real redaction flow.

Reference-only component: `ai-core/prism-visual-pipeline` is a self-contained face+PII demo (its own Qdrant-backed matching loop, InsightFace `buffalo_l`) kept in the repo as a harvested reference implementation. It is not wired into the production backend and is excluded from the counts in Section 4–5. `ai-core/audio-services` is likewise a superseded prototype of `audio-worker` , kept for reference only.

4. Worker Inventory — 12 Total

5 AI / ML MICROSERVICES One Python FastAPI container per modality, each with its own model stack.	7 BACKEND QUEUE / TIMER WORKERS Node.js processes sharing the backend image, driving BullMQ queues or interval sweeps.	13 CONTAINERS, FULL STACK 12 workers + 1 API process, per the staging compose file's own count.
--	---	--

4.1 Tier A — AI/ML Microservices (5)

face-worker core port 8001 Stateless face detection + ArcFace embeddings + bystander blur redaction.
image-pii-worker core, not optional port 8002 OCR + Presidio-based PII detection/masking on photos; whole image pipeline fails closed without it.
audio-worker opt-in port 8003 Diarization + transcription + PII flagging + mute-interval redaction + voice-embedding extraction.
text-worker core, always-on port 8004 Text PII analysis with character offsets + consent-aware span redaction.
video-worker opt-in port 8005 Face detection/tracking/sparse-embedding on video + box-schedule redaction execution.

4.2 Tier B — Backend Queue & Timer Workers (7)

Worker	Trigger	Role
<code>recognition.worker.js</code>	BullMQ queue: <code>face-recognition</code>	Runs face matching for a session against the Qdrant gallery once a session ends.
<code>redaction.worker.js</code>	BullMQ queue: <code>redaction-retry</code>	Retries photos that failed synchronous PII/face redaction.
<code>purge.worker.js</code>	BullMQ queue: <code>dsar-purge</code>	Executes DSAR erasure jobs and issues deletion certificates.
<code>itemAction.worker.js</code>	BullMQ queue: <code>dsar-item-action</code>	Executes per-item DSAR redact/delete/export-mark actions.

<code>export.worker.js</code>	BullMQ queue: <code>project-export</code>	Builds DSAR access-package ZIP exports.
<code>retention.worker.js</code>	Interval sweep (no queue)	Sweeps raw originals past their retention TTL; expires DSAR download packages.
<code>reaper.worker.js</code>	Interval sweep (no queue)	Requeues stalled photo/video jobs, times out stuck recognition jobs.

Excluded from the count as dead/prototype code, confirmed not referenced by any live compose file or backend module: `ai-core/audio-services` and `ai-core/prism-visual-pipeline`.

5. End-to-End Flows — 11 Total

A "flow" here means a complete journey a piece of content or a user action takes through the system, start to finish — not a single API call.

1. **Governance / project approval** — Data Owner drafts a project; DPO reviews purpose limitation; project is approved and its policy version frozen before any collection can begin.
2. **Collection session + consent + face enrollment** — Agent opens a session, subject authenticates via OTP, reads the notice, signs consent, and enrolls a 5-pose face capture (embedding encrypted at rest).
3. **Photo recognition & clustering** — On session end, a recognition job runs face-worker detection, clusters faces in-session, and lets the agent tag clusters to subjects.
4. **Photo redaction & consent-mapping** — face-worker blurs unconsented bystanders, image-pii-worker masks PII text, and only the redacted derivative is retained downstream.
5. **Ingest-to-golden-store** — Approved handoffs are written into the encrypted vault, retention clocks start, and only redacted derivatives feed analytics/search.
6. **Import (admin-initiated inbound)** — Admin-uploaded files for a named subject are PII-masked (no face detection, since no session/consent record exists for auto-blur).
7. **Voice enrollment** — Subject registers a voice print via ECAPA-TDNN embedding, encrypted at rest with per-subject limits.
8. **Audio capture, diarization & redaction** — Recording is diarized/transcribed/PII-flagged, speaker matched against the voice gallery, then muted at consent-driven intervals.
9. **Video capture, tracking & redaction** — Uploaded video is detected/tracked/embedded, a human tags clusters, then the worker executes the resulting blur schedule.
10. **Text/document PII analysis & redaction** — Pasted or uploaded text is scanned for PII spans and redacted according to per-span consent state.
11. **DSAR (Data Subject Rights)** — A rights request is triaged by the DPO, evidence is located across storage tiers, and export or purge is executed with a signed deletion certificate.

Two additional background sweeps exist as maintenance flows rather than content pipelines and are tracked separately: the **retention/TTL sweep** and the **stalled-job reaper/recovery sweep** (both **Tier B** workers above, Section 4.2).

6. Orchestration Layer

Component	Role
Redis + BullMQ	Message broker behind 5 named queues — <code>face-recognition</code> , <code>redaction-retry</code> , <code>dsar-purge</code> , <code>dsar-item-action</code> , <code>project-export</code> — mapping directly onto Flows 3, 4, and 11.
Qdrant (vector DB)	Per-session, ephemeral face and voice embedding galleries used for consent matching in Flows 3 and 8; collections are created at session start and destroyed at session end so no biometric vector persists beyond its intended scope.
Interval timers	<code>retention.worker.js</code> and <code>reaper.worker.js</code> run their own loops independent of Redis, rather than consuming a queue.
Synchronous HTTP calls	Audio, video, and text redaction (Flows 8–10) are direct request/response calls from the backend to their AI worker — no queue mediates them, unlike the image pipeline.

7. Design Principles Observed Across the Pipeline

- **No third-party inference in production.** Every model that runs in the live redaction path executes locally in its own container; the one cloud model found (GPT for name disambiguation) is explicitly confined to an offline demo tool.
- **Detect / Decide / Redact separation.** Every AI worker is stateless and makes no consent decisions — that logic lives solely in the backend, so a bug in an ML worker cannot silently expose unconsented data.
- **Fail closed, not open.** Workers that cannot complete a scan (e.g. `image-pii-worker`) return an explicit error rather than an empty "no PII found" result, so an outage can never be mistaken for a clean scan.
- **Irreversible redaction.** Blur, mute, and text-masking parameters are deliberately tuned to destroy signal (e.g. wide-kernel Gaussian blur, mosaic-then-blur) rather than merely obscure it, defeating deblurring/reconstruction attacks.
- **Shared embedding spaces enforced by design.** Where two workers must compare biometric vectors (`face-worker` ↔ `video-worker`), they are forced onto the identical model so cosine-similarity comparisons remain valid.